Core Matching Concept

The engine compares every transaction in Company A against every transaction in Company B, looking for opposing entries — A's debit should match B's credit (and vice versa).

The 6-Layer Waterfall

Matching runs top to bottom. Once a transaction is matched in any layer, it's removed from further consideration:

Layer	Logic	Confidence
1. Exact	Same amount + same date	100%
2. Date flex	Same amount + date within 7 days	90%
3. Rounding	Amount differs ≤ ₹5 + date within 7 days	85%
4. Tax (TDS)	Amount differs by a known tax rate (e.g. 8.47%, 10%, 18%) + date within 7 days	85%
5. Weighted score	Scores each pair on amount (50%) + date (30%) + ref (10%) + description (10%). Match if score ≥ 55%	Score-based
6. Partial settlement	Multiple small entries on one side sum up to one large entry on the other side	80%

How the Reconciliation System Works — Step by Step

What Is This App For?

Two companies do business together. Each company records the same transactions in their own books — but from opposite sides. Company A records a **payment** (debit), while Company B records a **receipt** (credit) for the same amount.

This app **automatically matches** those transactions between the two companies and finds any differences.

Step 1: Upload Files

You upload two Excel files exported from Tally:

- **File A** — one company's ledger (e.g., "SRV Ledger in Welcare Books")
- **File B** — the other company's ledger (e.g., "Welcare Ledger in SRV Books")

Step 2: Load & Clean the Data (

normalizer.py**)**

Tally Excel files have **junk rows** at the top — company name, address, date range, blank lines — before the actual data begins.

What happens:

1. **Find the real header row** — Scans the first 30 rows looking for keywords like "Date", "Debit", "Credit", "Particulars". The row with the most matches becomes the header.
2. **Merge split columns** — Tally splits the "Particulars" column into two: one has "To"/"By" prefix, the next has the actual name. These get combined into one column (e.g., "To" + "HDFC-CC" → "To HDFC-CC").
3. **Map column names** — The system recognizes various column names (e.g., "Vch No." = voucher number, "Particulars" = description) and renames them to standard names.
4. **Parse dates** — Converts text dates like "15-Apr-2024" into proper date values.
5. **Clean amounts** — Removes commas, currency symbols (₹, $) and converts to numbers.
6. **Filter junk rows:**

- Removes "Opening Balance" and "Closing Balance" rows
- Removes totals rows at the bottom (detected by dates before year 2000)
- Removes rows with zero amounts and no description

Result: A clean table with columns: date, debit, credit, description, voucher number.

Step 3: Match Transactions (

matcher.py**)**

The engine tries to match each transaction from Company A with a transaction from Company B. It goes through **6 layers**, from most confident to least:

Layer 1 — Exact Match ✅ (Confidence: 100%)

"Same amount, same date, opposite direction"

- A pays ₹1,50,000 on 5-Apr-2024 (debit)
- B receives ₹1,50,000 on 5-Apr-2024 (credit)
- **→ Perfect match!**

Layer 2 — Timing Difference 📅 (Confidence: 90%)

"Same amount, but dates differ by a few days"

- A records on 5-Apr, B records on 8-Apr (3 days apart)
- Amount is exactly the same
- **→ Match! Just a posting date difference.**

Layer 3 — Rounding Difference 🔢 (Confidence: 85%)

"Almost the same amount, difference ≤ ₹5"

- A: ₹10,003 and B: ₹10,000 (₹3 difference)
- **→ Match! Just rounding.**

Layer 4 — Tax Deduction (TDS) Pattern 💰 (Confidence: 85%)

"Amounts differ by a known tax percentage"

This is the most important layer for your Tally data. When Company B invoices ₹5,900 (base ₹5,000 + 18% GST), Company A pays only ₹5,400 because they deduct 10% TDS (₹500) on the base amount.

- B: ₹5,900 (invoice)

- A: ₹5,400 (payment after TDS)
- Difference: ₹500 = 8.47% of ₹5,900
- The system knows this is the standard TDS+GST pattern
- **→ Match! TDS deduction detected.**

The system checks against common tax rates: 1%, 2%, 5%, 7.5%, 8%, 8.5%, 10%, 15%, 18%, 20% (with ±1% tolerance).

Layer 5 — Weighted Score ⚖️ (Confidence: varies)

"Calculate an overall similarity score"

For remaining unmatched items, the system scores each possible pair:

- **Amount similarity** — 50% weight (how close are the amounts?)
- **Date proximity** — 30% weight (how close are the dates?)
- **Reference similarity** — 10% weight (do voucher numbers match?)
- **Description similarity** — 10% weight (do descriptions match?)

If the total score ≥ 55%, it's considered a match.

Layer 6 — Partial Settlement 🧩 (Confidence: 80%)

"One big payment = multiple small receipts"

- A pays ₹10,000 in one entry
- B received it as ₹4,000 + ₹3,000 + ₹3,000 (three entries)
- The system finds groups of 2–5 entries that sum up to the other side
- **→ Match! Partial settlement.**

Step 4: Classify Exceptions

Anything not matched becomes an **exception**:

- **"Missing in Company B"** — A has it, B doesn't
- **"Missing in Company A"** — B has it, A doesn't

In your data, the only exception is B's "TDS 23-24" entry (₹58,561) — a TDS credit that A doesn't record.

Step 5: Display Results

Screen	What it shows
Data Preview	The cleaned data from both files
Results Dashboard	KPIs: match rate, total matches, match types pie chart
Match Details	Every matched pair with dates, amounts, difference, match type
Exceptions	Unmatched transactions that need investigation

Your Data — What Actually Happened

Step	Result
File A loaded	17 real transactions (after removing headers, totals, balances)
File B loaded	18 real transactions
Exact matches	5 pairs (same amount + same date)
TDS matches	12 pairs (amounts differ by ~8.47% due to TDS deduction)
Exceptions	1 entry (B's TDS credit — no counterpart in A)
Match rate	Company A: **100%**, Company B: **94.4%**

Feedback submitted

Command Awaiting Approval